

IN-PLAY FOOTBALL ANALYTICS METHODOLOGY

Premier League Live Match Predictor

Continuous-Time Markov Chain Simulation & Dynamic Bayesian Blending

Quantitative Sports Analytics Research Group

July 29, 2026

Abstract

This document details a high-frequency predictive modeling pipeline for English Premier League in-play matches. Leveraging a comprehensive dataset of over 5,700 matches spanning from the 2011/12 through 2025/26 seasons, we model the continuous-time evolution of ball events across a 12-state space. In-play win/draw/loss outcome probabilities are forecasted via GPU-accelerated Monte Carlo simulations on a dynamically blended transition matrix, $\mathbf{Q}_{\text{active}}$, combining Bayesian-updated historical priors ($\mathbf{Q}_{\text{pre}}$), k -Nearest Neighbours historical trajectory matching ($\mathbf{Q}_{\text{KNN}}$), and real-time in-play Poisson transition rates ($\mathbf{Q}_{\text{live}}$). The pipeline operates within a strict < 200 ms latency budget to flag positive-expected-value (+EV) arbitrage opportunities against live bookmaker odds.

1 State Space & Historical Dataset

We ingest ball-event data covering 15 consecutive Premier League seasons ($\approx 5,700$ matches from 2011/12 to 2025/26). To reduce data dimensionality while capturing essential spatial and possession dynamics, we define a **12-state continuous-time Markov chain (CTMC)** state space $\mathcal{S}$:

- **10 Transient States:** Five discrete spatial pitch zones ($Z \in \{0, 1, 2, 3, 4\}$) crossed with the team currently holding possession ($P \in \{\text{Home}, \text{Away}\}$).
- **2 Terminal Absorbing States:** Goal states for either side (Goal_H and Goal_A).

Consequently, each infinitesimal generator matrix $\mathbf{Q}$ has dimensions 12×12 .

1.1 Temporal Exponential Decay Weighting

To account for long-term tactical evolution across 15 seasons without discarding historical sample size, we apply an exponential decay weight w_{season} to older season data:

$$w_{\text{season}} = e^{-\gamma \cdot \Delta s} \quad (1)$$

where Δs is the number of seasons elapsed relative to the current season, and $\gamma > 0$ is a configurable hyperparameter controlling the rate of historical discounting.

2 Dynamic Transition Matrix Formulation ($\mathbf{Q}_{\text{active}}$)

At any in-play match minute $t \in (0, 90]$, simulation is governed by an active generator matrix $\mathbf{Q}_{\text{active}}(t)$, formed as a dynamically weighted linear combination of three distinct component matrices:

$$\mathbf{Q}_{\text{active}}(t) = w_{\text{pre}}(t) \mathbf{Q}_{\text{pre}} + w_{\text{KNN}}(t) \mathbf{Q}_{\text{KNN}} + w_{\text{live}}(t) \mathbf{Q}_{\text{live}} \quad (2)$$

The blending weights are governed by continuous exponential decay curves with a half-life of 45 minutes, avoiding harsh step functions and ensuring smooth transition regime shifts as match information accumulates.

Dynamic Blending Weights Overview

- **Q_{pre} Weight:** Dominant at $t = 0$; decays rapidly as live evidence unfolds.
- **Q_{KNN} Weight:** Small initially, peaks in the mid-game (where trajectory similarity is most informative), and attenuates in late-game scenarios.
- **Q_{live} Weight:** Sparse and heavily discounted early; dominates in late-game stages as CTMC empirical rates stabilize.

2.1 Q_{pre} : Bayesian Prior Transition Matrix

The prior generator matrix bakes in league-wide structural baseline rates, home-field advantage, specific club identities, and pre-match strength differentials:

1. **Global Home/Away Baselines:** We aggregate state transition counts n_{ij} and holding times T_i across the entire 15-season database to build an overarching global Q matrix, extracting isolated baseline rates for teams playing at home versus away.
2. **Bayesian Conjugate Updating:** For a specific club playing at home, global home transition counts n_{ij}^{global} and holding times T_i^{global} are scaled by a concentration hyperparameter α and updated with the club's specific home observations ($n_{ij}^{\text{club}}, T_i^{\text{club}}$):

$$\lambda_{ij,\text{updated}} = \frac{\alpha n_{ij}^{\text{global}} + n_{ij}^{\text{club}}}{\alpha T_i^{\text{global}} + T_i^{\text{club}}} \quad (3)$$

An analogous update is performed when the club plays away. Here, α governs the regularization strength toward the Premier League average.

3. **Strength Rating Differential Scaling:** Incorporating standardized pre-match club strength ratings (e.g., Opta power ratings), each transition rate is adjusted via log-linear scaling:

$$\lambda_{ij,\text{final}} = \lambda_{ij,\text{updated}} \cdot \exp(\beta \cdot (\pm \Delta_{\text{rating}})) \quad (4)$$

where Δ_{rating} is the strength differential, and the sign is positive for advantageous transitions (e.g., progressive territorial advancement) and negative for adverse events (e.g., turnovers or disposessions).

2.2 Q_{KNN} : Nearest Neighbours Trajectory Matrix

To capture match-flow context, we construct 90 discrete databases—one for each minute bucket $t \in \{1, 2, \dots, 90\}$. In database t , all 5,700 historical matches are truncated at minute t .

For each truncated match, we compute and normalize a **5-dimensional feature vector**:

$$\mathbf{v}_t = \begin{bmatrix} \text{Field Tilt} \\ \text{Home Progression Ratio} \\ \text{Away Progression Ratio} \\ \text{Match Tempo} \\ \text{Goal Differential} \end{bmatrix} \quad (5)$$

For a live match at minute t , we evaluate its current vector $\mathbf{v}_t^{\text{live}}$, query database t , and extract the k -nearest neighbours via Euclidean distance. Q_{KNN} is derived by aggregating observed transition counts n_{ij} and holding times T_i from minute t to 90 across these k matches.

Distance-Based Penalty Scaling

The contribution weight $w_{\text{KNN}}(t)$ is penalized if the retrieved neighbours reside in a sparse region of the feature space. We scale the weight by the inverse or inverse-square of the cumulative Euclidean distance across the k neighbours:

$$w_{\text{KNN}}(t) \propto \frac{1}{\sum_{m=1}^k \|\mathbf{v}_t^{\text{live}} - \mathbf{v}_t^{(m)}\|_2^2}$$

2.3 $\mathbf{Q}_{\text{live}}$: Live CTMC Empirical Matrix

For the active match, we continuously aggregate observed ball transitions n_{ij}^{live} and state holding times T_i^{live} . Empirical Poisson transition rates are computed as:

$$\lambda_{ij}^{\text{live}} = \frac{n_{ij}^{\text{live}}}{T_i^{\text{live}}}, \quad \forall i \neq j \quad (6)$$

with diagonal elements set to $\lambda_{ii}^{\text{live}} = -\sum_{j \neq i} \lambda_{ij}^{\text{live}}$ to satisfy Markov generator conditions.

3 Real-Time Execution Pipeline (<200 ms Latency Budget)

To provide actionable probabilities for in-play betting and trading, the end-to-end simulation pipeline executes in under **200 milliseconds** upon receiving an event payload.

Pipeline Stage	Architectural Execution Details
1. Payload Ingestion	Parse streaming Opta ball-event feed; update live counts n_{ij}^{live} , holding times T_i^{live} , and current scoreline.
2. k-NN Query	Compute $\mathbf{v}_t^{\text{live}}$ and retrieve k -nearest historical trajectories from minute bucket t .
3. Weight Calculation	Evaluate continuous exponential half-life curves and distance penalty to form $\mathbf{Q}_{\text{active}}(t)$.
4. GPU Monte Carlo	Execute tensorized short-horizon Monte Carlo simulations from minute t to 90, initialized at current state and scoreline.
5. Market Alpha & eV	Devig live Bet365/Pinnacle odds, evaluate real-time Market RMSE, and flag +EV arbitrage discrepancies.

Table 1: End-to-End Real-Time Computational Pipeline

4 Engineering Safeguards & Numerical Stability

4.1 Dead-Time Capping

In-play interruptions (e.g., VAR reviews, prolonged injury treatments, substitution windows) distort Poisson holding-time estimates. We enforce a hard ceiling on individual state duration:

$$T_{i,\text{event}} \leftarrow \min(T_{i,\text{event}}, 15 \text{ seconds}) \quad (7)$$

preserving data integrity against unnatural lulls in match tempo.

4.2 Zero-Holding-Time Regularization

If a team has not yet visited state i during a live match ($T_i^{\text{live}} = 0$), raw division $\frac{n_{ij}}{T_i}$ fails. The engine implements **row-wise dynamic reallocation**:

- Unvisited states in $\mathbf{Q}_{\text{live}}$ automatically fall back to the corresponding row of $\mathbf{Q}_{\text{KNN}}$ or $\mathbf{Q}_{\text{pre}}$.
- Every row of each generator matrix is strictly verified and normalized such that $\sum_j \lambda_{ij} = 0$.

4.3 PyTorch Tensorized GPU Execution

Standard iterative Monte Carlo loops in Python suffer from CPU overhead and branching delays. To maintain latency within the < 200 ms budget:

- All Markov trajectory simulations execute as **pure PyTorch tensorized GPU bursts**.
- Branching logic (**if/else** conditionals) is eliminated via masked tensor operations and batch parallel matrix exponentiation/sampling.

5 Market Integration & Arbitrage Detection

In the final execution phase, the engine connects to live streaming bookmaker odds (e.g., Bet365, Pinnacle). The market integration module executes the following workflow:

1. **Devigging:** Removes bookmaker margin (overround) via Shin’s method or power-law normalization to extract true market-implied probabilities ($\mathbf{p}_{\text{market}}$).
2. **Real-Time Market RMSE:** Computes the root-mean-square error between model-predicted probabilities ($\mathbf{p}_{\text{model}}$) and market-implied probabilities over rolling windows.
3. **Alpha Discovery & +EV Flagging:** Identifies positive expected value (+EV) arbitrage opportunities where bookmaker quotes lag match dynamics. The $\mathbf{Q}_{\text{KNN}}$ component serves as the primary driver of alpha by recognizing structural historical patterns before they are reflected in consensus odds.